

Advanced Problem-Solving Lab – High-IQ Track

Activity H1 – Solve the “Hidden Problem” Behind the Stated Problem

Title: From Surface Problem to Real Problem

Goal

Train yourself to see the *real* problem, not just the stated one.

Instructions

You are given a “client statement”:

“Our exam system is too slow. We want a Python script to speed it up.”

1. Identify at least 3 possible *real* problems this statement might hide.

- Hidden problem 1: Inefficient algorithm
- Hidden problem 2: High latency due to network/database access
- Hidden problem 3: Resource limitation

2. For each hidden problem, state:

- What data would you need to confirm it?
 - Problem 1: Execution time profiling of function/algorithm
 - Problem 2: Network request log
 - Problem 3: System resource usage stats

3. Choose ONE hidden problem and re-frame it as a clear technical problem statement (2–3 lines):

New problem statement:

The exam system performance is degraded because its core processing algorithm has quadratic time complexity

4. Design a very small Python experiment that could validate or reject your reframed problem (pseudo-code is fine):

python

Experiment idea (what will you measure, on what data, how?)

```
def original_algorithm(data):
    result = 0
    for i in range(len(data)):
        for j in range(len(data)):
            result = data[i] + data[j]
    return result

def optimized_algorithm(data):
    return sum(data)**2
```


Activity H2 – Multiple Competing Solutions

Title: Design 3 Solutions Before Coding 1

Goal

Avoid fixation. Explore at least three distinct solution strategies.

Instructions

Problem:

"You receive a stream of log lines (text). You must detect 'suspicious' activity in real time."

1. Give three different solution ideas, each using a distinct main technique or data structure.

- Approach A (e.g., rule-based / regex / thresholds):

apply thresholds for severity

- Approach B (e.g., statistical / sliding window / counters):

maintain a sliding window of recent log entries

- Approach C (e.g., simple ML / anomaly detection concept):

Train a light weight anomaly - detection algorithm

2. For each approach, list pros and cons (at least one each).

- Approach A – Pros: fast Cons: require manual / rule maintenance
- Approach B – Pros: capture temporal patterns Cons: Need tuning of window size
- Approach C – Pros: Learn Cons: require training data

3. Choose one approach to implement first and justify:

I choose approach A because it provides quick transparent detection. minimal setup & is suitable for real time processing of text

4. Sketch high-level Python structure (modules/classes, not full code):

python

Modules / classes / key functions list

```
def parse_line(self, line):  
    pass
```

```
def match_rules(self, field):  
    pass
```

```
def evaluate(self, log_data):  
    return 'suspicious'
```


Activity H3 – Self-Constrained Challenge: "Elegant Code Only"

Title: Solve with Elegance (Clarity, Minimalism, Correctness)

Goal

Produce code that is not only correct, but also minimal and beautiful.

Problem

Implement a mini "pipeline" for processing a list of tasks:

- Each task has an ID, description, and status ("todo", "doing", "done").

• You must:

- Load tasks from a JSON file.
- Filter by status.
- Sort by ID.
- Print a clean summary.

Constraints (you impose on yourself)

1. Maximum 25 lines of *real* Python (excluding blanks and comments).
2. No function longer than 7 lines.
3. Names must be meaningful and short.

Steps

1. Design your data model in 2–3 lines:

2. Decide at least one thing you will *not* do to keep it minimal:

3. Write your code here (respecting constraints):

python

Elegant pipeline solution

4. Reflection:

- What did you have to sacrifice (if anything) to keep elegance?

to keep code minimal & elegant & modular input validation
focusing on core operation within the strict line limits

import json

Task = lambda d: dict(id=d["id"], desc=d["desc"], status=d["status"])

tasks = load('tasks.json')

filtered = filter(lambda task: task['status'] == 'todo', tasks)

sorted_tasks = sorted(filtered, key=lambda task: task['id'])

summary(sorted_tasks)

Activity H4 – Complexity and Scaling Thought Experiment

Title: How Will Your Solution Break at Scale?

Goal

Develop realistic intuition about performance and scaling.

Instructions

Take ANY non-trivial function or algorithm you have recently written (search, summarization, scoring, etc.).

1. Describe what it does, in one line:

merge sort is a divide & conquer algorithm

2. Estimate its time complexity in Big-O notation, and justify in 1–2 lines:

- Big-O: $O(n \log n)$

- Because: merge sort divides one array into halves

3. Scaling scenarios – imagine input sizes:

- $n = 10^3$ (1,000)
- $n = 10^6$ (1,000,000)
- $n = 10^9$ (1,000,000,000)

For each, predict: will it still be usable (Y/N) and why?

n	Usable (Y/N)	Reason (1 short sentence)
10^3	Y	sorting 1000 elements is relatively fast
10^6	Y	still manageable with modern computing power
10^9	N	sorting a billion elements with $O(n \log n)$ complexity

4. Suggest one change to improve scalability and briefly describe how it helps:

use a parallel sorting algorithm or distribute the sorting task across multiple processors to decrease overall time

Activity H5 – Adversarial Thinking: Break Your Own Design

Title: Attack Your Own Program

Goal

Develop robustness and defensive mindset.

Instructions

Choose one of your own projects or mini-tools (file reader, API client, CLI, etc.).

1. Write a short description of what it is supposed to do:

Program reads a file specified by the user & displays its contents

2. Now wear the hat of an attacker or malicious user.

List 5 ways to misuse / break it:

- Attack 1: Providing a non-existent file path
- Attack 2: feeding a very large file to cause memory issue
- Attack 3: using a file with special characters
- Attack 4: attempting to read a dictionary
- Attack 5: Trying to exploit path

3. For each attack, suggest one defence in plain language:

- Defence for attack 1: check if the file exists
- Defence for attack 2: Implement chunked & culling
- Defence for attack 3: Handle encoding & special characters
- Defence for attack 4: check if path is file, not a directory
- Defence for attack 5: Validate & sanitize input paths

4. Implement at least one of these defences in your code and paste the updated snippet:

python

Hardened code snippet:

```
import os

def safe_read_file(file_path):
    if not os.isfile(file_path):
        print("Error: not a valid file")
        return

    try:
        with open(file_path, 'r', encoding='utf-8') as file:
            for line in file:
                print(line.strip())
    except:
        print(f"An error occurred: {e}")
```


Activity H6 – Cross-Domain Transfer

Title: Apply Known Pattern to a New Domain

Goal

Practice lifting a known idea/pattern from one domain to a different one.

Instructions

Pick a pattern you already know well, e.g.:

- Sliding window
- Two-pointer technique
- Dynamic programming table
- Producer-consumer
- Map-reduce style
- Pipeline of filters

1. Name of pattern: Two-pointer technique

2. Original domain where you learned it (e.g., arrays, strings):

Arrays

3. New domain to apply it in (e.g., logs, time series, learning activities, medical data):

Medical data

4. Describe how you would adapt the pattern to this new domain, in 5–8 lines (can include pseudo-code):

use two-pointer to transfer a sorted list of appointment
start pointer 1 tracks the start of an appointment
pointer 2 tracks when pointer 1's start time < pointer 2's
end time

5. Optional: implement a small prototype showing this transfer:

python

Pattern transfer prototype

```
appointments = [(start1, end1), (start2, end2), ...]
sort appointments by start time
pointer 1 = 0
pointer 2 = 1
while pointer 2 < len(appointments):
    if appointment[pointer 1][1] > appointment[pointer 2][0]:
        pointer 1 += 1
    pointer 2 += 1
```


Activity H7 – Meta-Cognition: Your Personal Problem-Solving Manual

Title: Write Your Own "How I Solve Problems" Guide

Goal

Turn your tacit skills into an explicit, reusable system.

Instructions

1. List the first three things you usually do when you get a new problem (be honest):

- Step 1: Panic & Google it
- Step 2: Try random approaches
- Step 3: Ask a friend

2. Now, design your ideal 6–8 step process (what a "future you" would always do):

1. Define problem clearly
2. Break it into smaller part
3. Research known solutions.
4. Sketch potential approaches.
5. Prototype & test
6. Refine based on feedback
7. (optional) Document the solution
8. (optional) Share with others

3. For each step, write one question you will ask yourself:

- Step 1 question: What exactly is the problem asking?
- Step 2 question: Can I isolate the core issue?
- Step 3 question: Has anyone solved something similar?
- Step 4 question: What are the simplest possible solutions?
- Step 5 question: Does my prototype work as expected?
- Step 6 question: What can I improve?

4. Final instruction to yourself (one sentence):

"When I feel stuck, I will take a short break & revisit the problem definition"